

My Rust Notes

tyrone

June 18, 2026

Contents

1	Flow Control	2
1.1	If-else statement	2
1.2	match Expressions	3
1.3	Loops	4
1.3.1	While loops	4
1.3.2	For loops	4
1.3.3	loop	5
2	Macros and #[Derive]	5
3	User Defined Structures	7
3.1	Structs	7
3.1.1	Tuple Structs	7
3.2	Enums	8
3.3	static	9
3.4	const	9
4	References	10
4.1	Shared References	10
4.2	Exclusive References	11
5	Slices	11
6	Strings	12
7	Reference Validity	13
8	Return	13

9	Pattern Matching	14
9.1	Irrefutable Patterns	14
9.2	Destructuring Structs	14
9.3	Destructuring Enums	15
9.4	Let Control Flow	16
9.4.1	if let expression	16
9.4.2	while let statements	17
9.4.3	let else statement	18
10	Methods	19
11	Traits	21
11.1	Implementing traits	22
11.2	Supertraits	23
11.3	Associated Types	24
12	Generics	24
12.1	Generic Functions	25
12.2	Generic structs	26
12.3	Generic Structs with impl	26
12.4	Trait bounds - constraining what T can be	27
13	Closures	27
14	Ownership	30

1 Flow Control

1.1 If-else statement

if condition { code; } else if condition2 { code; } else { code; }

```
fn main() {
    let x = 10;
    if x == 0 {
        println!("zero");
    } else if x < 100 {
        println!("less than 100");
    } else {
        println!("more than 100");
    }
}
```

```
}  
}
```

An if statement may also be used as an expression.

```
fn main() {  
  
    let x = 10;  
    let size = if x < 20 { "small" } else { "large" };  
    println!("number size: {}", size);  
}
```

1.2 match Expressions

```
match value { value1 => code, value2 => code, value3 => code, _ => {  
    default, } }
```

```
fn main() {  
    let flag = true;  
    let val = match flag {  
        true => 1,  
        false => 0,  
    };  
  
    println!("The value of {flag} is {val}.");  
}
```

```
#[rustfmt::skip]  
fn main() {  
  
    let input = 'x';  
    match input {  
        'q'                => println!("Quitting"),  
        'a' | 's' | 'w' | 'd' => println!("Moving around"),  
        '0'..'9'            => println!("Number input"),  
    }  
}
```

```

key if key.is_lowercase() => println!("Lowercase: {key}"),
-                          => println!("Something else"),
  }
}

```

As you may see, '|' may be used as an **OR**.
 .. matches any number of items.
 1..5 is used to represent a range.
 _ is a wild card.

1.3 Loops

1.3.1 While loops

while condition { code; }

```

fn main() {
    let mut x = 0;

    while x < 10 {
        dbg!(10);
    }
}

```

1. *while let* patterns This allows binding new variables into scope when the pattern matches.

1.3.2 For loops

for x in firstValue..lastValue { code; }

```

fn main() {
    for x in 1..5 {
        dbg!(x);
    }

    for elem in [2, 4, 8, 16, 32] {

```

```

    dbg!(elem);
    }
}

```

1.3.3 loop

The loop statement just runs forever until it hits a break.

```

loop { code; }

```

```

fn main() {
    let mut i = 0;

    loop {
        dbg!(i);
        i += 1;
        if i == 10 {
            break;
        }
    }
}

```

2 Macros and #[Derive]

Macros are expanded during compilation into Rust code. They are indicated by a `!` at the end. Functionally, they are code that writes other code. Which is known as metaprogramming.

`#[derive]` is used to implement traits for structs and enums. Traits such as **Debug**, **Clone**, **Copy**, **PartialEq**, **Eq**, **Hash**, **Default**, and stuff like that. These traits are already built-in or imported.

```

#[derive(Debug, Clone, PartialEq)]
struct Point {
    x: f64,
    y: f64,
}

fn main() {

```

```

    let p = Point { x: 1.0, y: 2.0 };
    println!("{:?}", p);
}

```

`#[derive(Debug)]` allows you to print a value with `{:?}` or `{:#?}`. Use this with `println!`.

`#[derive(Clone, Copy)]` allows you to explicitly duplicate value using `.clone()` and then copy it somewhere else.

```

#[derive(Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

fn main() {

    let a = Point { x: 1.0, y: 2.0 };
    let b = a; // b copies a, not moved. This means a is still valid
    let c = a.clone(); // cloned
}

```

The trait `PartialEq` let's you compare using `==` and `!=`. The trait `Eq` requires `PartialEq` and signals that equality is total (every value equals itself). `FLoats` cannot implement `Eq` due to `NaN != NaN`.

Macros

```
println!("Hello, {}!", name);
print!("no newline");
eprintln!("error: {}", message);
format!("hello {}", name);
panic!("something went wrong");
todo!()
dbg!()
unreachable!()
unimplemented!()
concat!("hello", " ", "world");
file!()
vec!([1, 2, 3]);
matches!(val1, val2);
assert_eq!(val1, val2);
assert_ne!(val1, val2);
```

Function

```
Prints with a newline
Prints without a newline
Prints to stderr
Returns a String instead of printing
Immediately crashes with a message
Marks unfinished code, panics if reached
prints file/line/value then returns the value
Tells the compiler this point can't be reached
Essentially the same with todo!()
Concatenates strings at compile time
The current filename as &str
Creates a vector
Returns a bool depending on pattern match
Panics if not equal
Panics if equal
```

3 User Defined Structures

3.1 Structs

Similar to the ones in C and C++. Can store multiple variables with different data types. Unlike in C++, there is no inheritance between structs.

```
struct Person {
    name: String,
    age: u8,
}
```

3.1.1 Tuple Structs

If the names of the variables are unimportant, tuple structs are used instead.

```
struct Coordinate(i32, i32);
```

Used for single-field wrappers called newtypes.

```

struct PoundsOfForce(f64);
struct Newtons(f64);

fn compute_force() -> PoundsOfForce {
    //...
}

fn convert_force(force: PoundsOfForce) -> Newtons {
    //...
}

```

3.2 Enums

This defines one type with a fixed set of possible values.

```

#[derive(Debug)]
enum Direction {
    Left,
    Right,
}

#[derive(Debug)]
enum playerMove {
    Pass, // Simple variant
    Run(Direction), // Tuple variant
    Teleport { x:u32, y: u32 }, //Struct variant
}

fn main() {

    let dir = Direction::Left;
    let player_move: playerMove = playerMove::Run(dir);
    println!("On this turn: {player_move:?}");
}

```


Direction is now a type with variants. The variants being *Left* or *Right*. *Direction* has two values: *Direction::Left* and *Direction::Right*.

3.3 static

Static variables will live during the whole execution of the program and cannot be moved.

```
static MESSAGE: &str = "Hello, World.";

fn main() {
    println!("{MESSAGE}");
}
```

3.4 const

Constants are evaluated at compile time to generate const values. I think?

```
const DIGEST_SIZE: usize = 3;
const FILL_VALUE: u8 = calculate_fill_value();

const fn calculate_fill_value() -> u8 {
    if DIGEST_SIZE < 10 { 42 } else { 13 }
}

fn compute_digest(text: &str) -> [u8; DIGEST_SIZE] {
    let mut digest = [FILL_VALUE; DIGEST_SIZE];

    for (idx, &b) in text.as_bytes().iter().enumerate() {
        digest[idx % DIGEST_SIZE] = digest[idx % DIGEST_SIZE].wrapping_add(b);
    }
    digest
}

fn main() {
    let digest = compute_digest("Hello");
    println!("digest: {digest:?}");
}
```

```
}
```

4 References

You may have many shared references but only one exclusive reference.

4.1 Shared References

A reference is a way to access another value without taking ownership of the value. This is called 'borrowing'. Shared references are read-only, the data cannot be changed.

```
fn main() {  
    let a = 'A';  
    let b = 'B';  
  
    let mut r: &char = &a;  
    dbg!(r);  
  
    r = &b;  
    dbg!(r);  
}
```

A reference value is made with the `&` operator. The `*` operator dereferences a reference.

A shared reference to a type T has the type $\&T$.

Rust can auto-dereference in many cases.

```
let mut s = String::from("hello");  
let c = &mut s;  
  
c.push_str(" world"); // no * needed, Rust auto-derefs for method calls  
println!("{}", s); // "hello world"
```

4.2 Exclusive References

Exclusive references are known as mutable references. They allow you to change the value they are referring to.

```
fn main() {  
  
    let mut point = (1, 2);  
    let x_coord = &mut point.0;  
    *x_coord = 20;  
    println!("point: {point:?}"); // Output=> point: (20, 2)  
}
```

They have type **&mut T**.

Very similar to pointers. Create a variable and point it to the reference using **&mut**. Then make changes to that variable using **&mut** and the changes made to the new variable will reflect in the original variable.

5 Slices

A slice let's you view into a larger collection.

```
fn main() {  
  
    let a: [i32; 6] = [10, 20, 30, 40, 50, 60];  
    println!("a: {a:?}");  
  
    let s: &[i32] = &a[2..4];  
    println!("s: {s:?}");  
}
```

Slices borrow data.

If the slice starts at index 0, you may drop the starting index so it will be **&a[..a.len()]**

The same is true for the last index, so `&a[2..a.len()]` is the same as `&a[2..]`.

To make a slice of the full array, use `&a[..]`

You cannot append to a slice since it doesn't own the backing buffer.

6 Strings

`&str` is a slice of UTF-8 encoded bytes. This is a string slice, an immutable reference to UTF-8 encoded string data stored in a block of memory. `String` is an owned buffer of UTF-8 encoded bytes.

```
fn main() {  
  
    let s1: &str = "World";  
  
    let mut s2: String = String::from("Hello ");  
  
    s2.push_str(s1);  
    println!("s2: {s2}"); //Outputs: Hello World  
  
    let s3: &str = &s2[2..9];  
    println!("s3: {s3}"); // Outputs: 'llo Wor'  
}
```

The `String` data type of Rust is a wrapper around a vector of bytes.

`String::new()` creates a new, empty string to which data can be added using the `.push()` and `.push_str()` methods.

`String::from()` creates an owned, growable `String` from a literal string.

```
let s1 = String::from("hello");  
  
let s2 = "hello".to_string(); //same as s1  
  
let s3 = String::new() //empty string
```

The `.push()` method appends a single character (`char`).

The `.push_str()` method appends a string slice (`&str`).

```

let mut s = String::from("hello");

s.push('!'); // this is fine
s.push("!"); //this is &str, not char so this will lead to an error

s.push_str(" world");

```

Other useful String methods

```

let mut s = String::from("hello world");

s.contains("world");           // outputs true
s.replaces("world", "rust");   // returns new String "hello rust"
s.to_uppercase();              // "HELLO WORLD"
s.len();                       // 11 bytes since it uses 11 characters
s.is_empty();                  // outputs false
s.split(" ").collect::<Vec<_>>(); // ["hello", "world"]

```

7 Reference Validity

In Rust, references are always safe to use due to certain rules.

One of the rules is that references can never be null.

References can't also outlive the data they point to.

8 Return

The **return** command does exist but is generally only used early in the program.

Just type the variable you want the program to output at the end but don't end it with a semicolon.

Rust will always return a value that doesn't end with semicolon.

Unless you use the **return** keyword, then end with a semicolon.

9 Pattern Matching

9.1 Irrefutable Patterns

```
fn takes_tuple(tuple: (char, i32, bool)) {

    let a = tuple.0;
    let b = tuple.1;
    let c = tuple.2;

    let (a, b, c) = tuple; //This does the same as the block above

    let (_, b, c) = tuple; //Ignores the first element, binds only the second and th

    let (.., c) = tuple; //Ignores everthing but the last element
}

fn main() {
    takes_tuple(('a', 777, false));
}
```

Irrefutable means that the patters will always match the value on the RHS.

Variable names are patterns that always match and bind the matched value into a new variable with that name.

Patterns are type-specific.

`_` is a pattern that always matches any value, discarding the matched value.

`..` allows you to ignore multiple values at once.

9.2 Destructuring Structs

```
struct Move {
    delta: (i32, i32),
    repeat: u32,
}

#[rustfmt::skip]
```

```

fn main() {
    let m = Move { delta: (10, 0), repeat: 5};

    match m {
        Move { delta: (0, 0), ..}      => println!("Standing still"),
        Move { delta: (x, 0), repeat }  => println!("{repeat} step x: {x}"),
        Move { delta: (0, y), repeat: 1 } => println!("Single step y: {y}"),
        _                               => println!("Other move"),
    }
}

```

`x` and `y` aren't defined so running this code will just lead to a compile error.

9.3 Destructuring Enums

You may wonder what even is destructuring.

Destructuring is unpacking a data structure into its individual components in one field.

```

//This is unpacking each value of the tuple one by one.
let point = (3.0, 4.0);
let x = point.0;
let y = point.1;
println!("{}", x, y);

//This is destructuring; just takes one line.
let point = (3.0, 4.0);
let (x, y) = point;
println!("{}", x, y);

```

Anyway, back to Enums.

```

enum Shape {
    Circle(f64),
    Rectangle(f64, f64),
}

```

```

        Triangle { base: f64, height: f64 },
    }

let shape = Shape::Circle(5.0);

match shape {
    Shape::Circle(r) => println!("Circle with radius {}", r),
    Shape::Rectangle(w, h) => println!("Rectangle {}x{}", w, h),
    Shape::Triangle { base, height } => println!("Triangle base={} height={}", base,
    _ => println!("Some other shape."),
}

```

9.4 Let Control Flow

9.4.1 if let expression

This lets you execute different code depending on whether a value matches a pattern.

```

use std::time::Duration;

fn sleep_for(secs: f32) {

    let result = Duration::try_from_secs_f32(secs);

    if let Ok(duration) = result {
        std::thread::sleep(duration);
        println!("slept for {duration:?}");
    }
}

fn main() {
    sleep_for(-10.0);
    sleep_for(0.8);
}

```


Unlike **match**, **if let** does not have to cover all branches.

This is used to handle **Some** values when working with **Options**.

Unlike **match**, **if let** does not support guard clauses for pattern matching.

With an **else** clause, this can be used as an expression.

9.4.2 while let statements

This repeatedly tests a value against a pattern.

```
fn main() {  
  
    let mut name = String::from("Comprehensive Rust");  
  
    while let Some(c) = name.pop() {  
        dbg!(c);  
    }  
}
```

In this weird ass example to print out each character of a string in reverse order, **String::pop** returns **Some(c)** until the string is empty, after which it will return ***None**.

Wait, what the heck does **Some()** do?

Some() is a variant of the **Option** enum.

```
enum Option<T> {  
    Some(T),    // there IS a value, and this is it  
    None,      // there is NO value  
  
}
```

T can be any type: **Some(5)**, **Some("hello")**, **Some(Point{...})**, and such.

Rust has no **null**. If a value is absent, the type system will need you to handle it.

```
fn find_user(id: u32) -> Option<String> {
    if id == 1 {
        Some(String::from("Alice")) // found a value
    } else {
        None // There really is no value
    }
}
```

Anyway, **while** **let** loops will keep going so long as the value matches the pattern.

This cannot be used as an expression because they may have no value if the condition is false.

9.4.3 let else statement

If you're just matching a pattern and returning from the function, use **let else**. The **else** case must diverge into **return**, **break**, or **panic** - anything but falling off the end of the block.

```
fn hex_or_die_trying(maybe_string: Option<String>) -> Result<u32, String> {
    let Some(s) = maybe_string else {
        return Err(String::from("got None"));
    };

    let Some(first_byte_char) = s.chars().next() else {
        return Err(String::from("got empty string"));
    };

    let Some(digit) = first_byte_char.to_digit(16) else {
        return Err(String::from("not a hex digit"));
    };

    Ok(digit)
}
```

So, **Result** is the same with **Option** but for error handling.

```

enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),      // success, has a value
    Err(E),     // failure, has an error
}

fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("cannot divide by zero"))
    } else {
        Ok(a / b)
    }
}

match divide(10.0, 2.0) {
    Ok(result) => println!("quotient: {}", result),
    Err(e) => println!("oops: {}", e),
}

```

E type can be anything: a string message, **io::Error**, **ParseError**.

```

Result<f64, String>; // If successful, expect a float. Otherwise, a string.

Result<u32, io::Error>;

Result<(), ParseError>; // T is () - success has no value, just that it 'works'

```

10 Methods

```

    // Copyright 2023 Google LLC
    // SPDX-License-Identifier: Apache-2.0

    #[derive(Debug)]
    struct CarRace {
        name: String,
        laps: Vec<i32>,
    }

    impl CarRace {
        // No receiver, a static method
        fn new(name: &str) -> Self {
            Self { name: String::from(name), laps: Vec::new() }
        }

        // Exclusive borrowed read-write access to self
        fn add_lap(&mut self, lap: i32) {
            self.laps.push(lap);
        }

        // Shared and read-only borrowed access to self
        fn print_laps(&self) {
            println!("Recorded {} laps for {}: ", self.laps.len(), self.name);
            for (idx, lap) in self.laps.iter().enumerate() {
                println!("Lap {idx}: {lap} sec");
            }
        }

        // Exclusive ownership of self (covered later)
        fn finish(self) {
            let total: i32 = self.laps.iter().sum();
            println!("Race {} is finished, total lap time: {}", self.name, total);
        }
    }

    fn main() {
        let mut race = CarRace::new("Monaco Grand Prix");
        race.add_lap(70);
        race.add_lap(68);
        race.print_laps();
    }

```

```

    race.add_lap(71);
    race.print_laps();
    race.finish();
    // race.add_lap(42);
}

```

The **self** argument specifies the object the method acts on (the 'receiver').

It could be **&self** which borrows the object from the caller using a shared and immutable reference. The object can be used again later.

It could be **&mut self** which borrows the object from the caller using a unique and mutable reference. The object can be used again later.

It could be **self** which takes ownership of the object from the caller. The method becomes the new owner of the object. The object will be deallocated or 'dropped' when the method returns, unless ownership is explicitly transmitted.

It could be **mut self** which is the same as above, taking new ownership, but the method can mutate the object.

No receiver is also an option. This is usually used to create constructors that are called **new** by convention.

Methods are called on an instance of a type such as a struct or enum. The first parameter represents the instance as **self**.

self, the method receiver, is abbreviated for **self: Self**.

Self is a type alias for the type the **impl** block is in and can be used elsewhere within the block.

Basically, *self* refers to the instance. *Self* refers to the type.

11 Traits

Traits are used to abstract over types, similar to interfaces.

```

trait Pet {

    fn talk(&self) -> String;

    fn greet(&self);
}

```

Traits defines a number of methods that types must have in order to implement the trait.

11.1 Implementing traits

```
trait Pet {
    fn talk(&self) -> String;

    fn greet(&self) {
        println!("Oh, what's your name? {}", self.talk());
    }
}

struct Dog {
    name: String,
    age: i8,
}

impl Pet for Dog {
    fn talk(&self) -> String {
        format!("Woof, my name is {}!", self.name)
    }
}

fn main() {
    let fido = Dog { name: String::from("Fido"), age: 5 };
    dbg!(fido.talk());
    fido.greet();
}
```

To implement **Trait** for **Type**, use an **impl Trait** for **TType { .. }**

block.

Unlike in Go interfaces, just having matching methods is not enough: a **Cat** type with a **talk()** method would not automatically satisfy **Pet**. Unless it is in an **impl Pet** block.

11.2 Supertraits

A trait can require that types implementing it also implement other traits, called *supertraits*.

```
// Copyright 2024 Google LLC
// SPDX-License-Identifier: Apache-2.0

trait Animal {
    fn leg_count(&self) -> u32;
}

trait Pet: Animal {
    fn name(&self) -> String;
}

struct Dog(String);

impl Animal for Dog {
    fn leg_count(&self) -> u32 {
        4
    }
}

impl Pet for Dog {
    fn name(&self) -> String {
        self.0.clone()
    }
}

fn main() {
    let puppy = Dog(String::from("Rex"));
    println!("{}", puppy.name(), puppy.leg_count());
}
```

11.3 Associated Types

These are placeholder types that are supplied by the trait implementation.

```
#[derive(Debug)]
struct Meters(i32);

#[derive(Debug)]
struct MetersSquared(i32);

trait Multiply {

    type Output = MetersSquared;

    fn multiply(&self, other: &Self) -> Self::Output;
}

impl Multiply for Meters {
    type Output = MetersSquared;

    fn multiply(&self, other: &Self) -> Self::Output {
        MetersSquared(self.0 * other.0)
    }
}

fn main() {
    println!("{}", Meters(10).multiply(&Meters(20)));
}
```

Associated types are sometimes called 'output types'.

It is important to note that the implementer, not the caller, chooses this type.

12 Generics

Generics lets you abstract algorithms or data structures over the types used or stores.


```

fn pick<T>(cond: bool, left: T, right: T) -> T {
    if cond { left } else { right }
}

fn main() {
    println!("pick a number: {:?}", pick(true, 222, 333));
    println!("pick a string: {:?}", pick(false, 'L', 'R'));
}

```

This can be used for stuff like sorting or a binary tree.

Generics can reduce code duplication; letting you write code that works for any type. This is essentially what **Options**<Some, None> and **Result**<Ok, Err> are.

12.1 Generic Functions

Instead of:

```

fn largest_i32(list: &[i32]) -> i32 {
    let mut largest = list[0];
    for &item in list {
        if item > largest { largest = item; }
    }
    largest
}

fn largest_f64(list: &[f64]) -> f64 {
    let mut largest = list[0];
    for &item in list {
        if item > largest { largest = item; }
    }
    largest
}

```

You can just:

```
fn largest<T: PartialOrd>(list: &[T]) -> T {
    let mut largest = list[0];
    for &item in list {
        if item > largest { largest = item }
    }
    largest
}
```

```
largest(&[1, 2, 3, 4]); //i32
largest(&[1.0, 2.0, 3.0, 4.0]); //f64
```

12.2 Generic structs

This is what **Option** and **Result** are.

```
struct Wrapper<T> {
    value: T,
}

let w1 = Wrapper { value: 42 };
let w2 = Wrapper { value: "hello" };
let w3 = Wrapper { value: 3.14 };
```

12.3 Generic Structs with impl

```
struct Wrapper<T> {
    value: T,
}

impl<T> Wrapper<T> {
    fn new(value: T) -> Self {
        Self { value }
    }

    fn get(&self) -> &T {
        &self.value
    }
}
```

```
}
```

12.4 Trait bounds - constraining what T can be

Without bounds, T could be anything. Trait bounds say "T must implement this trait"

```
use std::fmt::Display;

// T must implement Display trait if it is to be printed
fn print_value<T: Display>(value: T) {
    println!("{}", value);
}

// T must implement Display and Clone traits to use this function
fn print_and_clone<T: Display + Clone>(value: T) {
    let copy = value.clone();
    println!("{}", copy);
}
```

Generics	Meaning
<T>	placeholder for any type
<T: Trait>	T must implement this trait
<T: Trait + Trait>	T must implement these traits
<T, U>	multiple type placeholders

13 Closures

Anonymous functions that can store variables or be passed to other functions.
It is enclosed in `| |`.

```
fn add(a: i32, b: i32) -> i32 { a + b }
```

```
let add = | a, b| a + b;      // same as above.
```

Regular functions can't use variables from the surrounding scope. But closures can. This is called *capturing*. The closure "closes over" the variable.

```
let x = 10;

let add_x = |n| n + x; // captures x from outside
println!("{}", add_x(5)); // outputs 15
```

Closures can capture by **referencing &T**, this is the default and just reads the value.

```
let name = String::from("Alice");

let greet = || println!("hello {}", name); // borrows name variable

greet();

println!("{}", name); // name still usable
```

By mutable reference **&mut T**:

```
let mut count = 0;

let mut increment = || count += 1; //mutably borrows count

increment();

increment();

println!("{}", count); // outputs 2
```

By ownership with **move**:

```
let name = String::from("Alice");

let greet = move || println!("hello {}", name); // takes ownership of name

greet(); // name variable is gone. "Alice" has been moved into the closure.
```

Examples:

```
let numbers = vec![1, 2, 3, 4, 5];

let doubled: Vec<i32> = numbers.iter()
    .map(|n| n * 2)
    .collect(); // [2, 4, 6, 8, 10]

let evens: Vec<i32> = numbers.iter()
    .filter(|n| *n % 2 == 0)
    .collect(); // [2, 4]

let first_even = numbers.iter()
    .find(|n| *n % 2 == 0); // Some(2)

let sum = numbers.iter().fold(0, |acc, n| acc + n); // 15

let result: Vec<i32> = numbers.iter()
    .filter(|n| *n % 2 == 0)
    .map(|n| n * 10)
    .collect(); // [20, 40]

// As a function argument

fn apply(f: impl Fn(i32) -> i32, value: i32) -> i32 {
    f(value)
}

let double = |n| n * 2;
```

```
println!("{}", apply(double, 5)); // 10
```

There are **three** traits for closure.

<code>Fn</code>	borrows immutably, can be called many times.
<code>FnMut</code>	borrows mutably, can be called many times.
<code>FnOnce</code>	takes ownership, can only be called once.

14 Ownership

Every variable in Rust has **exactly one owner at a time**.

When the owner goes out of scope, the value is automatically dropped (freed).

Ownership can be transferred, this is called a move.

```
let s1 = String::from("hello");  
let s2 = s1; // ownership of "hello" moves from s1 to s2.  
  
println!("{}", s1); // Error since s1 no longer owns any value.  
  
println!("{}", s2); // "hello"
```